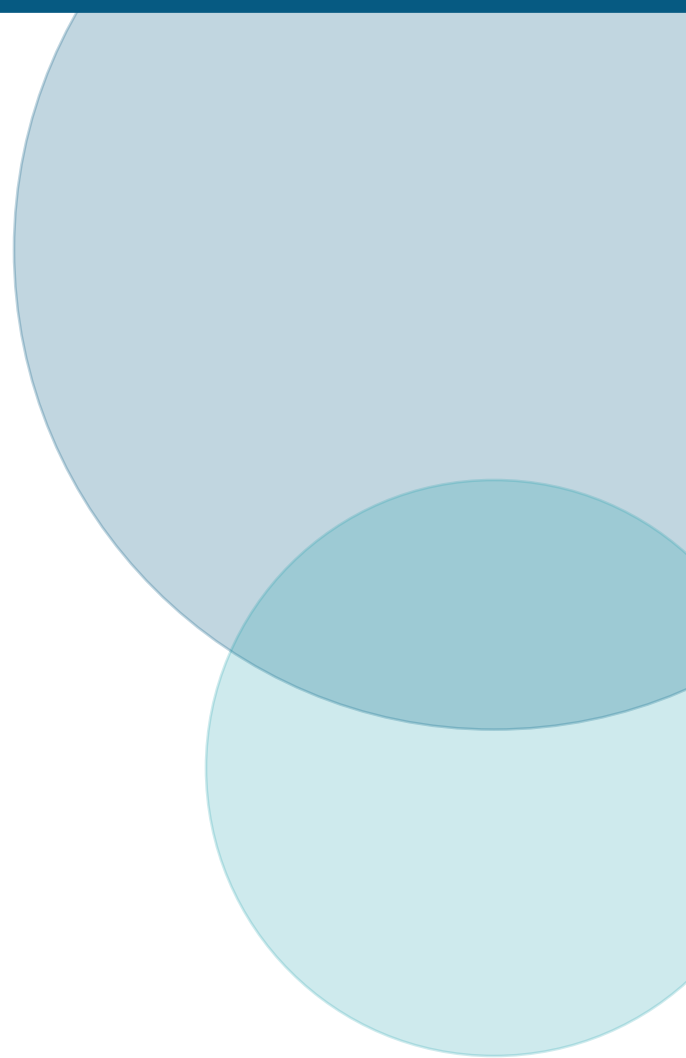


Building Immersive Experiences with Web Technologies

Matteo Valoriani



AR vs MR vs VR vs XR

AT A GLANCE



AR

AUGMENTED
REALITY



ADDS TO
REALITY



VR

VIRTUAL
REALITY



REPLACES
REALITY

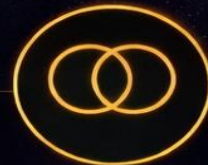


MR

MIXED
REALITY



MERGES
REALITIES



XR

EXTENDED
REALITY



EVERYTHING
TOGETHER

Native XR vs Web XR

Aspect	Native XR	Web XR
Distribution	App store, SDK install	URL — instant access
Performance	Direct hardware access, optimized	Browser overhead, improving fast
Cross-platform	Per-platform builds required	One codebase, all devices
Update cycle	Store approval, user must update	Deploy instantly, always latest
Friction	Download → install → open	Click link → experience

Why WebXR?

0

Installs Required

Share a link. Users are inside the experience in seconds — no app stores, no downloads, no barriers.

1

Codebase for All

Desktop, mobile, headsets — a single web app runs everywhere the browser does.

∞

Iteration Speed

Deploy in seconds. Rapid prototyping, A/B testing, continuous improvement without gating.



Reduced Friction

~50% of users abandon before completing a native app install flow.

WebXR Device API Overview

A W3C specification that provides access to VR and AR hardware through the browser.

Browser Support

- Chrome — full support (desktop + Android)
- Edge — full support (Chromium-based)
- Meta Quest Browser — optimized for Quest
- Samsung Internet — partial support

Current Limitations

- No stable support in Safari or Firefox
- Hand tracking behind experimental flags
- Feature parity varies across devices
- Limited access to system-level XR features

WebXR ARCHITECTURE

How WebXR Works

The browser mediates between your JavaScript application and the XR hardware.



Immersive Sessions

immersive-vr and immersive-ar modes provide full XR rendering on a headset or AR-capable device.

Inline Sessions

inline mode renders 3D content in the browser window without entering XR mode — ideal for previews and fallbacks.

The Rendering Loop

WebXR uses a frame loop similar to `requestAnimationFrame`, managed by the XR runtime.

```
function onXRFrame(time, frame) {  
  const session = frame.session;  
  const pose = frame.getViewerPose(refSpace);  
  
  if (pose) {  
    const glLayer = session.renderState.baseLayer;  
    gl.bindFramebuffer(gl.FRAMEBUFFER, glLayer.framebuffer);  
  
    for (const view of pose.views) {  
      const viewport = glLayer.getViewport(view);  
      gl.viewport(viewport.x, viewport.y,  
                 viewport.width, viewport.height);  
      // Render scene for this eye  
      renderScene(view.projectionMatrix,  
                  view.transform.inverse.matrix);  
    }  
  }  
  session.requestAnimationFrame(onXRFrame);  
}
```


Key Concepts

Reference Spaces

- viewer — head-locked
- local — seated
- local-floor — standing
- bounded-floor — room scale
- unbounded — world scale

Pose & Tracking

- XRPose encodes position (vec3)
- and orientation (quaternion)
- of headset, controllers, or hands
- relative to a reference space.

Frame Loop

- Each XRFrame provides XRView
- objects (one per eye) with
- projection and view matrices
- for stereo rendering.

Input Sources

Controllers (6DoF)

Position + rotation in space.
Buttons, triggers, thumbsticks.
Most mature input method.

Stable

Gaze Pointer

Direction-based pointing from
head orientation. Used on 3DoF
devices or as fallback.

Stable

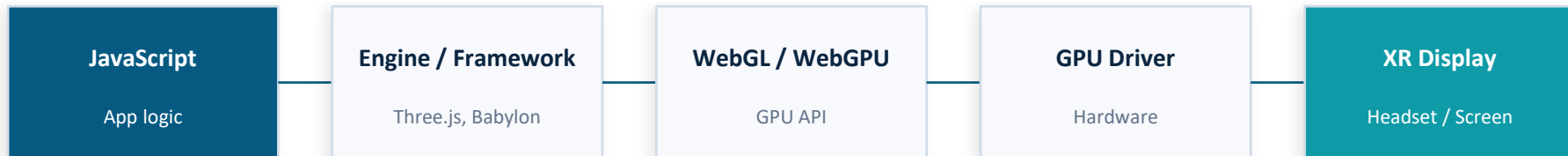
Hand Tracking

25-joint articulated hand model.
Gesture recognition for pinch,
grab, point.

Experimental

The Pipeline

From your code to the user's eyes



Frameworks abstract WebGL/WebGPU — focus on the experience, not shader plumbing.

Introduction to WebGL

WebGL is a JavaScript binding for OpenGL ES 2.0/3.0, providing GPU-accelerated 3D graphics in the browser.

What It Is

- The only standard for hardware-accelerated 3D in browsers
- Supported in all modern browsers
- Foundation of every WebXR experience today

Limitations

- Very low-level API — verbose boilerplate
- Manual shader writing (GLSL)
- Manual buffer and state management
- No compute shaders

The Move to WebGPU

WebGPU is the next-generation GPU API for the web, inspired by Vulkan, Metal, and DirectX 12.

Advantages over WebGL

- Compute shaders for GPGPU workloads
- More efficient render pipeline
- Better multi-threading potential
- Modern shader language (WGSL)

Current Status

- Shipping in Chrome since v113
- Three.js WebGPU renderer in development
- Babylon.js WebGPU support active
- Not yet formally integrated with WebXR spec

Three.js

The most popular 3D library for the web

- Huge community & ecosystem (90k+ GitHub stars)
- Scene graph: Scene → Camera → Mesh → Material
- Built-in WebXRManager for XR sessions
- Extensive examples and documentation
- Rich plugin ecosystem (postprocessing, controls)

```
const scene = new THREE.Scene();
const camera = new THREE.PerspectiveCamera(
  70, w/h, 0.1, 100
);
const renderer = new THREE.WebGLRenderer({
  antialias: true
});

renderer.xr.enabled = true;
document.body.appendChild(
  VRButton.createButton(renderer)
);

renderer.setAnimationLoop(() => {
  renderer.render(scene, camera);
});
```

Babylon.js

A complete 3D engine by Microsoft

- Online Playground & built-in Inspector
- PBR materials out of the box
- Integrated physics engines (Havok, Cannon)
- First-class WebXR via WebXRExperienceHelper
- Node Material Editor for visual shaders

```
const engine = new BABYLON.Engine(canvas);
const scene = new BABYLON.Scene(engine);
const camera = new BABYLON.FreeCamera(
  "cam", new BABYLON.Vector3(0,1.6,0), scene
);

const xr = await scene
  .createDefaultXRExperienceAsync({
    floorMeshes: [ground]
  });

engine.runRenderLoop(() => {
  scene.render();
});
```

Three.js vs Babylon.js

Criteria	Three.js	Babylon.js
Philosophy	Library — pick what you need	Engine — batteries included
Learning curve	Gentle start, deep ceiling	Steeper, more structured
XR support	Good, via WebXRManager	Excellent, first-class helpers
Tooling	Community-driven	Inspector, Playground, NME
Community	Very large (90k+ ★ GitHub)	Large (22k+ ★ GitHub)
Best for	Flexibility, prototyping	Structured projects, performance

Scene Graph



Core Objects

- Scene — root container
- Camera — defines the viewpoint
- Renderer — draws to canvas
- Mesh — Geometry + Material
- Light — illuminates the scene

Hierarchy Matters

- Child objects inherit parent transforms
- Use Group to organize related objects
- Transform order: scale → rotate → translate
- `scene.add()` / `parent.add(child)`

Geometries & Materials

- BoxGeometry, SphereGeometry, PlaneGeometry
- CylinderGeometry, TorusGeometry, RingGeometry
- BufferGeometry — custom vertex data
- ExtrudeGeometry — 3D from 2D paths

Materials

- MeshBasicMaterial — flat, no lighting
- MeshStandardMaterial — PBR (roughness/metalness)
- MeshPhysicalMaterial — clearcoat, transmission
- ShaderMaterial — custom GLSL shaders

```
// PBR Material
const mat = new THREE.MeshStandardMaterial({
  color: 0x2D5BE3,
  roughness: 0.4,
  metalness: 0.6,
  envMap: envTexture,
});

// Custom BufferGeometry
const geo = new THREE.BufferGeometry();
const verts = new Float32Array([
  -1,-1,0,  1,-1,0,  1,1,0,
  -1,-1,0,  1, 1,0, -1,1,0,
]);
geo.setAttribute('position',
  new THREE.BufferAttribute(verts, 3)
);
const mesh = new THREE.Mesh(geo, mat);
scene.add(mesh);
```

Loading 3D Models

Three.js provides loaders for all major formats. GLTFLoader is the standard for web.

Other loaders: FBXLoader · OBJLoader · TextureLoader · CubeTextureLoader (environment maps)

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';
import { DRACOLoader } from 'three/addons/loaders/DRACOLoader.js';

const dracoLoader = new DRACOLoader();
dracoLoader.setDecoderPath('/draco/');

const loader = new GLTFLoader();
loader.setDRACOLoader(dracoLoader);

loader.load(
  'models/robot.glb', (gltf) => {
    const model = gltf.scene;
    model.position.set(0, 0, -2);
    model.scale.setScalar(0.5);
    scene.add(model);

    // Access animations
    const mixer = new THREE.AnimationMixer(model);
    gltf.animations.forEach((clip) => {
      mixer.clipAction(clip).play();
    });
  },
```

Lights, Shadows & Camera

Light Types

- AmbientLight — uniform fill, no direction
- DirectionalLight — sun-like, casts shadows
- PointLight — omnidirectional (light bulb)
- SpotLight — cone-shaped with angle+penumbra
- HemisphereLight — sky + ground gradient

Camera for XR

- PerspectiveCamera: fov, aspect, near, far
- In XR mode, runtime overrides camera matrices

```
renderer.shadowMap.enabled = true;
renderer.shadowMap.type =
  THREE.PCFSoftShadowMap;

const dirLight = new THREE.DirectionalLight(
  0xffffff, 1
);
dirLight.position.set(5, 10, 5);
dirLight.castShadow = true;

// Shadow quality
dirLight.shadow.mapSize.width = 2048;
dirLight.shadow.mapSize.height = 2048;
dirLight.shadow.camera.near = 0.5;
dirLight.shadow.camera.far = 50;

scene.add(dirLight);

mesh.castShadow = true;
floor.receiveShadow = true;
```

Animation & Interaction

Animation System

- AnimationMixer — drives animations on a model
- AnimationClip — keyframe data
- AnimationAction — play, pause, crossfade
- Call mixer.update(delta) every frame

Raycaster for Interaction

- Cast ray from pointer into 3D scene
- Returns intersected objects with distance
- Works with mouse, touch, XR controller

```
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();

function onPointerDown(event) {
  pointer.x = (event.clientX /
    window.innerWidth) * 2 - 1;
  pointer.y = -(event.clientY /
    window.innerHeight) * 2 + 1;

  raycaster.setFromCamera(pointer, camera);
  const hits = raycaster.intersectObjects(
    scene.children, true
  );

  if (hits.length > 0) {
    const obj = hits[0].object;
    obj.material.color.set(0xff0000);
  }
}

renderer.domElement.addEventListener(
  'pointerdown', onPointerDown
);
```

WebXR HIGH-LEVEL FRAMEWORKS

Web AR Approaches

Marker-based

Recognizes predefined visual patterns (QR codes, fiducial markers) to anchor 3D content.

- Reliable when marker is visible
- Requires printed marker
- Limited spatial awareness

AR.js

ARToolKit

Markerless (SLAM)

Detects surfaces and tracks the environment without predefined targets.

- More natural user experience
- Higher computational cost
- Relies on WebXR Hit Test or proprietary SDK

8th Wall

WebXR AR

AR.js

Open-source library for marker-based AR in the browser, built on ARToolKit + Three.js/A-Frame.

github.com/AR-js-org/AR.js · MIT License · 5.9k stars · Integrates with A-Frame and Three.js

Performance

Targets 60 fps on mobile. Pure JavaScript — no plugin or install.
Requires WebGL and WebRTC support.

Browser-based

Works on Android, iOS and desktop.
Delivered via URL — no App Store, no installation. HTTPS + camera permission required.

Open Source

MIT licensed. Built on three.js and jsartoolkit5. A-Frame wrapper enables declarative HTML-based AR scenes.

Three AR tracking modes: Image Tracking — Marker Tracking — Location-based AR

Created by Jerome Etienne, now maintained by the AR.js org on GitHub.
The library uses jsartoolkit5 for tracking and can render content with either Three.js or A-Frame.

3 Tracking Modes

01 — Image Tracking

Detects 2D images via the camera. Overlays 3D content, video or images on the recognized target. Uses Natural Feature Tracking (NFT).

02 — Marker Tracking

Uses predefined markers (Hiro, Kanji) or custom .patt patterns. Also supports barcode markers encoding numeric IDs. Fast and stable in controlled environments.

03 — Location-based AR

Anchors AR content to real-world GPS coordinates. Content appears at fixed geographic positions, scaling with distance from the user.

Example — Marker Tracking (A-Frame)

```
<script src="aframe.js"></script>
<script src="aframe-ar.js"></script>

<a-scene embedded arjs>
  <a-marker preset="hiro">
    <a-box position="0 0.5 0"
      color="#0E9AA7">
    </a-box>
  </a-marker>
  <a-entity camera></a-entity>
</a-scene>
```

Point camera at Hiro marker → 3D box appears in AR. HTML only, no app.

AR in ~10 lines of HTML

AR.js

Open-source library for marker-based AR in the browser, built on ARToolKit + Three.js/A-Frame.

Pros

- Completely free and open source
- Works on mobile without app install
- Simple A-Frame integration
- Location-based AR mode available

Cons

- Marker-based only (no SLAM)
- Tracking can be unstable
- Limited visual quality
- Active maintenance varies

Known Limitations

AR.js was a pioneering project, but shows its age.

Sparse maintenance

Last significant update dates back several years. Open issues accumulate with slow response. The project is no longer actively driven.

Limited community support

Documentation is incomplete and partially outdated. Few recent tutorials exist. Stack Overflow coverage is thin compared to alternatives.

Technically dated

One of the first WebAR libraries — important historically, but the architecture has not kept pace. Newer libraries (MindAR, 8thWall) offer better tracking accuracy and richer APIs.

Context

AR.js was one of the first libraries to bring AR to the browser (2017).

At the time, running AR in a mobile browser at 60 fps was a significant achievement.

Today the landscape has moved on. MindAR and 8thWall offer better image tracking; WebXR is now a W3C standard.

8th Wall

Commercial platform for high-quality markerless Web AR experiences.

Features

- SLAM-based world tracking
- Image tracking & face effects
- Lighting estimation
- Cross-browser including Safari

Trade-offs

- Paid licensing model
- Vendor lock-in risk
- Proprietary SDK
- High quality but closed ecosystem



A-Frame — Make WebVR

Web framework for building 3D, AR and VR experiences with HTML

- Built on [three.js](#) (r173) with WebXR + WebGL support
- Declarative HTML with [Entity-Component-System](#) architecture (no JavaScript needed)
- Cross-platform: Vive, Rift, Quest, desktop, mobile
- Started by **Mozilla VR** (2015), now community-maintained, MIT License, 300+ contributors, Selected for [2024 GitHub Accelerator](#)
- Great for quick prototypes and demos

```
<html>
  <head>
    <script src="https://aframe.io/releases/1.7.1/aframe.min.js"></script>
  </head>
  <body>
    <a-scene>
      <a-box position="-1 0.5 -3" color="#4CC3D9"></a-box>
      <a-sphere position="0 1.25 -5" color="#EF2D5E"></a-sphere>
      <a-sky color="#ECECEC"></a-sky>
    </a-scene>
  </html>
```

GITHUB STATS

17.5k Stars

4.2k Forks

v1.7.1 Latest

Entity-Component-System Architecture

Composition over inheritance — mix and match reusable components to define objects

Entity

Container objects — the base of all objects in the scene. Without components, they render nothing, similar to empty `<div>`s.

```
<a-entity>
```

Component

Reusable modules that attach appearance, behavior, and functionality to entities. Like plug-and-play for objects.

```
position="0 1 -3"
```

System

Global scope and services for classes of components. Handles logic while components act as data containers.

```
AFRAME.registerSystem()
```

Built-in Components

[geometry](#)[material](#)[light](#)[animation](#)[camera](#)[raycaster](#)[shadow](#)[sound](#)[gltf-model](#)[tracked-controls](#)[cursor](#)[fog](#)[background](#)[ar-hit-test](#)[hand-tracking](#)

HTML Primitives

[a-box](#)[a-sphere](#)[a-cylinder](#)[a-plane](#)[a-sky](#)[a-circle](#)[a-ring](#)[a-torus](#)[a-cone](#)[a-image](#)[a-video](#)[a-text](#)

Ecosystem & Developer Tools

Community components, Visual Inspector, AR/VR platform support, and v1.7 features

Visual Inspector

- Built-in 3D inspector similar to browser DevTools
- Unity-like interface, open with `Ctrl+Alt+I`
- Inspect entities, tweak components, debug scenes live

Community Components

- Hundreds of npm packages: physics, particle systems, multiuser, oceans, teleportation, speech recognition
- Drop-in via HTML: `<a-entity ocean>`

AR & VR Platform Support

- WebXR spec support since v1.0 (2019)
- AR mode: ARCore + ARKit, hand tracking, hit-test
- Graceful fallback: desktop mouse + keyboard if no HMD

v1.7 Highlights

- Post-processing support (immersive + flat)
- ES modules + import maps, WebGPU/TSL support
- Logitech MX Ink tracked pen integration

A-Frame + AR.js Integration

AR.js uses A-Frame as its primary rendering layer, wrapping AR functionality into custom HTML components. The A-Frame version of AR.js enables marker tracking, image tracking, and location-based AR through declarative HTML attributes on `<a-scene>` elements.

```
<a-scene arjs="trackingMethod: best; sourceType: webcam;">
```

A-Frame — Pros & Cons

Pros

- Lowest entry barrier — HTML devs start immediately
- Entity-Component-System is flexible
- Large registry of community components
- Great for quick prototypes and demos

Cons

- Custom logic requires Three.js internals
- Performance overhead from DOM sync
- Debugging can be opaque
- Less suited for large-scale production apps

React Three Fiber

A React renderer for Three.js

- Every mesh, light, camera = JSX element with props and state
- Hooks: useFrame, useThree, useLoader
- Leverages entire React ecosystem
- Familiar to front-end developers
- Scalable codebase with component reuse

```
import { Canvas } from '@react-three/fiber'
import { useRef } from 'react'

function Box() {
  const ref = useRef()
  useFrame(() => {
    ref.current.rotation.x += 0.01
  })
  return (
    <mesh ref={ref}>
      <boxGeometry />
      <meshStandardMaterial
        color="orange"
      />
    </mesh>
  )
}

export default function App() {
  return (
    <Canvas>
      <ambientLight />
      <Box />
    </Canvas>
  )
}
```

React XR

Adds WebXR capabilities to React Three Fiber with dedicated hooks and components.

Key Features

- Controller & hand tracking components
- useXR(), useController() hooks
- Interaction layers with raycasting
- Teleportation and locomotion helpers

```
import {
  XR,
  Controllers,
  Hands,
  useXR,
  useController
} from '@react-three/xr'

export default function App() {
  return (
    <Canvas>
      <XR>
        <Controllers />
        <Hands />
        <ambientLight />
        <Floor />
        <Scene />
      </XR>
    </Canvas>
  )
}
```

React-based XR — Pros & Cons

Pros

- Modern, composable architecture
- Scalable codebase with component reuse
- Full React ecosystem (state, routing, testing)
- Familiar to front-end developers

Cons

- Steep curve: React + Three.js + R3F
- React reconciler overhead in hot paths
- Smaller XR-specific community than A-Frame
- Abstractions can obscure low-level control

Babylon.js XR System

Babylon provides an integrated, opinionated XR system with minimal setup.

Built-in Features

- WebXRDefaultExperience — session + teleport + controllers
- WebXRFeaturesManager — hand tracking, hit test, anchors
- Pointer selection & near interaction
- GUI panels in 3D space

Why Consider It

- Everything built-in — no plugin hunting
- Robust testing pipeline
- Strong performance characteristics
- Microsoft backing and enterprise adoption

Framework Comparison

	A-Frame	R3F + React XR	Babylon.js	Three.js
Paradigm	Declarative HTML	React JSX	Imperative	Imperative
Audience	Beginners, designers	React developers	Engine devs	Low-level devs
XR Support	Good	Good	Excellent	Manual setup
Performance	Moderate	Good	Very good	Best
Scalability	Limited	High	High	Depends
Learning Curve	Low	High	Med-High	Medium

WebXR OnMobile

Input Types in WebXR

Controllers (6DoF)

Full positional + rotational tracking. Buttons, triggers, thumbsticks, haptic feedback.

Production ready

Hand Tracking

25 joints per hand. Articulation model enables pinch, grab, point gestures without controllers.

Growing support

Eye Tracking

Experimental. Potential for foveated rendering and gaze-based UI intent detection.

Future

Interaction Patterns

Techniques

- Gaze-based — point with head direction, dwell timer confirms
- Raycasting — virtual ray from controller hits objects
- Gesture — pinch to select, grab to move, point to navigate
- Direct manipulation — near-field touch and grab

UX Best Practices

- Always provide visual feedback for hover and selection
- Make affordances clear — what can be interacted with?
- Avoid "gorilla arm" — no sustained raised arms
- Support multiple input fallbacks gracefully

WebXR on Mobile Devices

On smartphones, WebXR runs without a headset. The phone screen becomes both viewport and primary input device.

How It Works

- Browser requests an immersive-ar session
- Camera feed shown as passthrough background
- Phone acts as a "magic window" into the AR scene
- Gyroscope + accelerometer provide 6DoF tracking
- WebXR Hit Test detects surfaces for placement

Key Differences from Headset

- Single viewport (no stereo rendering)
- No controllers — touch is primary input
- Limited FOV (phone screen size)
- Thermal throttling is aggressive
- Works on Chrome Android + Samsung Internet
- iOS: No WebXR — use Model Viewer / AR Quick Look

Touch Interaction in Mobile WebXR

Without controllers, the touch screen is the interaction layer. WebXR exposes touch as a transient-pointer input source.

How Touch Maps to XR

- Tap — fires select event (= controller trigger)
- Tap & hold — selectstart / selectend for drag
- Touch point → ray into 3D scene (raycasting)
- Pinch-to-zoom — must implement manually
- Two-finger rotate — custom gesture for objects

```
session.addEventListener('select', (event) => {
  const source = event.inputSource;
  const frame = event.frame;
  const refSpace = renderer.xr.getReferenceSpace();

  const pose = frame.getPose(
    source.targetRaySpace, refSpace  );

  if (pose) {
    const origin = pose.transform.position;
    const dir = new THREE.Vector3(0, 0, -1)
      .applyQuaternion(pose.transform.orientation);
    raycaster.set(
      new THREE.Vector3(
        origin.x, origin.y, origin.z
      ), dir
    );
    const hits = raycaster
      .intersectObjects(objects);
    if (hits.length > 0) {
      handleObjectTap(hits[0]);
    }
  }
});
```

Tap-to-Place & Gestures

```
const hitTestSource = await session
  .requestHitTestSource({ space: viewerSpace });

// In the frame loop:
const hits = frame
  .getHitTestResults(hitTestSource);
if (hits.length > 0) {
  const hit = hits[0];
  const pose = hit.getPose(refSpace);
  reticle.visible = true;
  reticle.matrix.fromArray(
    pose.transform.matrix
  );
}

// On tap → place object
session.addEventListener('select', () => {
  if (reticle.visible) {
    const clone = model.clone();
    clone.position.setFromMatrixPosition(
      reticle.matrix
    );
    scene.add(clone);
  }
});
```

Custom Gesture Handling

- Pinch — scale placed objects up/down
- Two-finger rotate — spin on Y axis
- Swipe — cycle between models or UI panels
- Long press — context menu or delete mode

Implement via `touchstart`, `touchmove`, `touchend` on a DOM overlay. The WebXR DOM Overlay API enables HTML UI on top of the AR view.

WebXR Viewer

Web Component

`<model-viewer>`

`<rtf-viewer>`

Google <model-viewer>

The simplest way to display 3D models on the web — one HTML tag, full 3D + AR support.

```
<!-- Include the component -->
<script type="module" src=
  "https://ajax.googleapis.com/ajax/libs/
    model-viewer/3.5.0/model-viewer.min.js">
</script>

<!-- That's it -->
<model-viewer
  src="robot.glb"
  ios-src="robot.usdz"
  alt="3D Robot"
  ar
  ar-modes="webxr scene-viewer quick-look"
  camera-controls
  touch-action="pan-y"
  auto-rotate
  shadow-intensity="1"
  style="width:100%; height:400px;"
>
</model-viewer>
```

What You Get for Free

- Interactive 3D viewer — orbit, zoom, pan
- AR button: Android Scene Viewer + iOS AR Quick Look
- Responsive & accessible out of the box
- Lazy loading, poster image, auto-rotate
- PBR rendering with environment lighting
- Annotations (hotspots) for product details
- Animation playback controls

<model-viewer> — Features & Use Cases

Advanced Features

- Hotspots — annotate points with HTML labels
- Variants — switch glTF material variants
- Animation — play/pause/scrub embedded clips
- HDR environment maps, shadow control
- Programmatic orbit, zoom, FOV via JS API
- Poster placeholder while model loads

E-commerce & Configurators

Product viewers with AR try-out. Variant switching for colors and options. No framework needed.

Documentation & Education

Embed interactive 3D diagrams in docs or LMS. Works in any CMS that allows HTML.

Portfolio & Showcase

Artists and architects display 3D work with a single tag — no Three.js knowledge required.

Google <model-viewer> — Limits

<model-viewer> is a Google web component for embedding 3D models — with some limitations.

- Display of GLTF/GLB, OBJ, FBX, PLY, STL models
- HDR environments and customizable backgrounds
- Interactive hotspots with HTML content
- 3D animations with controller

- Semplice da usare, ma poche interazioni
- 1 solo modello alla volta
- Poche opzioni di personalizzazione

<RTF-VIEWER>

Google <model-viewer> with much more options and multi-model support

What You Get for Free

- Same feature of Google <model-viewer>
- Use many features of React Three Fiber, but with a simple component easy to integrate everywhere
- CSS and DOM Events to customize the interaction
- Multi-model support
- Possibility to differentiate AR vs Browser Mode

WebXR Component:

<https://go2.is/webxrviewer>

WebXR Demo:

<https://srvrfrm2fi.xyz/webxrviewer>

Integration

Two files need to be included in the page (script and style):

1

Include CSS

```
<link rel="stylesheet" href="rtf-viewer.css">
```

2

Include JS

```
<script type="module" src="rtf-viewer.js"></script>
```

3

Add tag

```
<rtf-viewer version="0" settings="{...}"></rtf-viewer>
```

Settings: General Structure

The "settings" attribute is a JSON object with the following sections:

<code>models</code>	Array of models and groups
<code>environment</code>	HDR environment
<code>camera</code>	FOV, near, far
<code>outline</code>	Border effect
<code>animationsSettings</code>	Autoplay, controller
<code>bounds</code>	Auto-fit camera

<code>draco</code>	Model decompression
<code>materials</code>	Material configuration
<code>xr</code>	Augmented Reality
<code>Lights</code>	Key, fill, back light
<code>Background</code>	Type, color, image

Settings: Models (basic props)

Each element of the "models" array has the following properties:

Property	Type	Description
"type"	model	Element type
"format"	string	"gltf", "obj", "fbx", "ply", "stl"
"mainFile"	string	3D File URL
"label"	string	Label text
"relatedFiles"	object	Related files: { filename: url }
"position"	[x, y, z]	Transform
"rotation"	[x, y, z]	Rotation
"scale"	[x, y, z]	Scale
"isVisible"	boolean	Visibility

Settings: Models (materials)

Each element of the "models" array has the following properties:

Property	Type	Description
"transparent"	boolean	Enable transparency
"color"	string	Color override (hex)
"metalness"	integer	Override metalness (0–1)
"roughness"	integer	Override roughness (0–1)
"wireframe"	boolean	Wireframe visualization

Settings: Models (AR)

Each element of the "models" array has the following properties:

Property	Type	Description
"arPosition"	[x, y, z]	AR Transform
"arRotation"	[x, y, z]	AR Rotation
"arScale"	[x, y, z]	AR Scale
"xr.visibleOnlyInXr"	boolean	Set a model visible only in XR mode
"xr.disableOnTapToPlace"	boolean	Exclude from tap-to-place

Settings: Models (Animations)

Each model has an "animation" object:

Property	Type	Default	Description
"name"	string	""	Animation name
"delay"	number	0	Start delay (seconds)
"timeScale"	number	1	Speed (1 = normal)
"loop"	boolean	true	Loop playback
"reverse"	boolean	false	Reverse playback

Settings: Models (Hotpoints — Marker)

Each model can have an array of "hotpoints":

Property	Type	Description
"id"	string	Unique identifier
"position"	[x, y, z]	Position relative to the model
"title"	string	Title on marker
"description"	string	Can be an "html" text

Settings: Environment

The "environment" object is available with the following properties:

Property	Type	Description
"enabled"	boolean	Enable an environment
"preset"	string	"apartment", "city", "dawn", "forest", "lobby", "night", "park", "studio", "sunset", "warehouse"
"files"	string	Custom HDR file URL (overrides preset)
"background"	boolean	Use environment as background
"blur"	float	Background blur (0–1)
"modulateColor"	string	Environment color modulation ("#FFFFFFF")

Settings: Camera & Bounds (auto-fit)

Properties related to "camera" and "bounds" (auto-positioning):

Property	Type	Description
"camera.fov"	number	Camera Field of View (e.g. 75)
"camera.near"	float	Near clipping plane (0.1)
"camera.far"	number	Far clipping plane
"initialCameraPosition"	[0,0,0]	Initial camera position
"initialOrbitTarget"	[0,0,0]	Initial orbit target
"bounds.enabled"	boolean	Automatically position camera
"bounds.margin"	number	Space around the model in the scene view
"bounds.cameraPosition"	[x,y,z]	Pre-bounds position, start with higher z e.g. [0, 0, 1000]

Settings: Lights

Properties related to lights in the scene (not in AR):

Property	Type (default)	Description
"keyLightIntensity"	float (3)	Key light intensity
"fillLightIntensity"	float (0.5)	Fill light intensity
"backLightIntensity"	float (1.5)	Back light intensity
"ambientLightIntensity"	float (1)	Ambient light intensity
"useLightsOnTarget"	boolean	5 directional lights aimed at models

Settings: Background

Properties related to the background (as an alternative to the environment):

Property	Type	Description
"backgroundType"	string	Background type: "transparent", "color", "image", "environment"
"backgroundColor"	string	Background color "#RRGGBB" or "#RRGGBBAA"
"backgroundImage"	string	Background image URL

Background types:

"environment"	Uses the HDR environment as background
---------------	--

"color"	Solid color fill
---------	------------------

"image"	Background image URL
---------	----------------------

"transparent"	Transparent background (compositing)
---------------	--------------------------------------

Settings: Outline & Draco

Properties for outline (post-processing) and Draco (decoder for compressed glTF):

Property	Type	Description
"outline.enabled"	boolean	Enable outline effect around models
"outline.edgeStrength"	number	Edge strength
"outline.edgeWidth"	number	Edge width
"draco.enabled"	boolean	Enable Draco decoder
"draco.path"	string	URI path to the decoder

Settings: XR — Augmented Reality

All properties are nested under the "xr" object in settings:

Property	Type	Default	Description	Property	Type	Default	Description
<code>"xr.enabled"</code>	boolean	false	Enable AR features	<code>"xr.showTranslateBtn"</code>	boolean	true	Show translation button
<code>"xr.enterArText"</code>	string	"Enter AR"	Entry button text	<code>"xr.freeManipulationEnabled"</code>	boolean	true	Free manipulation (drag, pinch)
<code>"xr.exitArText"</code>	string	"Exit AR"	Exit button text	<code>"xr.tapToPlaceLookAtCamera"</code>	boolean	false	Keep model looking at camera
<code>"xr.showHitTestBtn"</code>	boolean	true	Show "tap to place" button	<code>"xr.lookAroundText"</code>	string	"LOOK AROUND."	Text during floor scanning
<code>"xr.hitTestBtnText"</code>	string	"Tap to Place"	"Tap to place" button text	<code>"xr.tapToPlaceText"</code>	string	"TAP TO PLACE"	Text during model placement
<code>"xr.showRotateBtn"</code>	boolean	true	Show rotation button	<code>"usdzUrl"</code>	string	""	URL to .usdz file for iOS

Settings: Animations

Global animation settings properties:

Property	Type	Description
"animationsSettings.autoplay"	boolean	Autoplay animation on load
"animationsSettings.autoplayDelay"	number	Autoplay delay (seconds)
"animationsSettings.enableController"	boolean	Show animation controller
"animationsSettings.autoCloseController"	boolean	Hide animation controller with a click outside the model

Custom Events: Emitted by the Viewer

The viewer emits DOM events listenable with `addEventListener`:

Event	Description
"rtfviewer-load"	WebGL Canvas created and loaded
"rtfviewer-modelchange"	A model is loaded
"rtfviewer-allmodelsrendered"	All models rendered
"rtfviewer-model-clicked"	Click on a model (or empty area)
"rtfviewer-model-over"	Mouse over a model
"rtfviewer-model-out"	Mouse out of a model
"rtfviewer-camera-update"	Camera position updated
"rtfviewer-ar-started"	AR session started
"rtfviewer-ar-ended"	AR session ended
"settingschange"	Settings updated

Custom Events: Received by the Viewer

The viewer also listens to the following external events:

Event	Description
"rtf-update-settings"	Update settings at runtime
"rtf-update-selected-model"	Select a model by code
"rtfviewer-camera-sync"	Change camera position

```
window.dispatchEvent(new CustomEvent("rtf-update-settings", {  
  detail: { settings: newSettings }  
}));
```


CSS Classes: AR

Selector	Description
"#xr-ar-btn"	Button Enter / Exit AR
".enter-ar-btn"	State outside AR
".exit-ar-btn"	State inside AR
"#xr-controls-btns-container"	Container AR buttons
".xr-control-btn"	Single control button
".xr-control-btn.selected"	Active control button
"#xr-placer-container"	"Tap to place" message container
".look-around-text"	"Look around" text
".look-around-img"	Initialization animated gif

Full Configuration Example

```
<rtf-viewer version="0" settings='{
  "models": [{
    "type": "model",
    "format": "glTF",
    "mainFile": "https://example.com/model.glb",
    "scale": [1, 1, 1],
    "hotpoints": [{
      "id": "hpl",
      "position": [1, 1, 0],
      "title": "Info",
      "description": "<p>Detail</p>"
    }],
    "animation": { "name": "Idle", "loop": true }
  }],
  "environment": { "preset": "studio" },
  "bounds": { "enabled": true, "margin": 1.2 },
  "xr": { "enabled": true }
}'}></rtf-viewer>
```

ASSETS & CONTENT PIPELINE

Mobile vs Headset

Aspect	Mobile Smartphone	Standalone Headset (Quest)
GPU	Mobile-class, shared with OS	Dedicated XR GPU (Snapdragon XR2)
Thermal limit	Aggressive throttling after minutes	Better cooling, longer sustained perf
Refresh rate	60 fps typical	72–120 fps
Rendering	Single viewport	Stereo (2× viewports)
Poly budget	~50–100k triangles	~300–750k triangles
Input	Touch, device orientation	6DoF controllers, hand tracking

Performance Bottlenecks

CPU

JavaScript logic, physics simulation, animation updates, scene graph traversal.

GPU

Draw calls, shader complexity, fill rate, stereo rendering (2× viewports).

Network

Asset weight, initial load time, streaming latency for dynamic content.

Target: 72–90 fps sustained. Dropping below causes motion sickness and breaks immersion.

3D Asset Formats

glTF / GLB

The "JPEG of 3D" — Khronos Group standard. Compact binary, PBR materials, animations, skinning. The web standard.

Recommended

USDZ

Apple's format for AR Quick Look on iOS/macOS. Required for Safari AR experiences and Apple ecosystem.

Apple ecosystem

Legacy (FBX, OBJ)

Common in DCC tools. Useful as interchange formats during production, but convert to glTF before deployment.

Not for web

Asset Optimization

Polycount

Reduce geometry via decimation and retopology.
Target: <100k triangles for mobile, <500k for headset.

Texture Compression

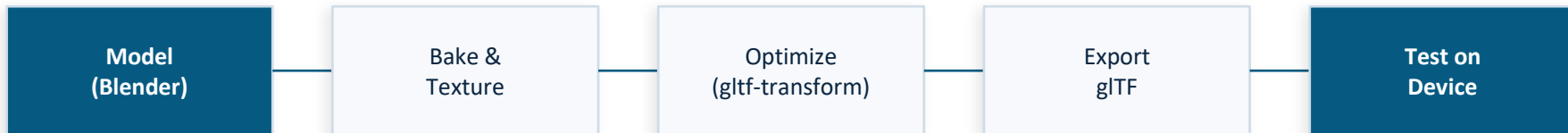
Use KTX2 with Basis Universal for GPU-compressed textures. Draco compression for geometry data.

Level of Detail (LOD)

Switch mesh resolution based on camera distance. Critical for scenes with many objects.

Every MB saved on assets translates directly to faster load times and higher frame rates — especially on mobile.

Tools & Pipeline



Key Tools

- Blender — modelling, texturing, glTF export
- gltf-transform — CLI optimize, compress, merge
- Sketchfab — 3D asset repository & web viewer
- Reality Composer — Apple USDZ authoring

Best Practices

- Always export as GLB (single binary file)
- Apply Draco + KTX2 compression
- Test on lowest-spec target device
- Automate pipeline with CI/CD scripts



AMAZ3D

**A fast and automatic tool
for 3D applications**

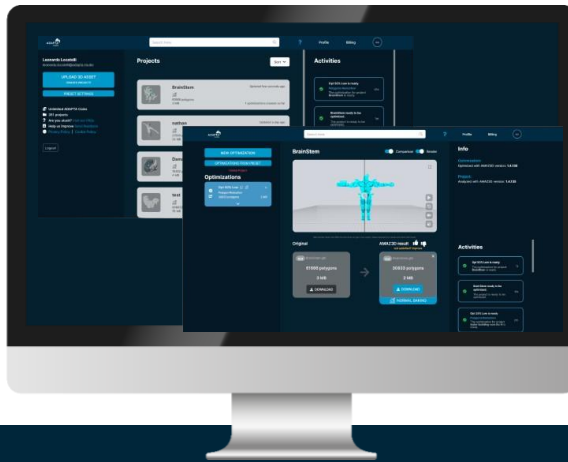
www.adapta.studio

A PRODUCT OF ADAPTA-STUDIO

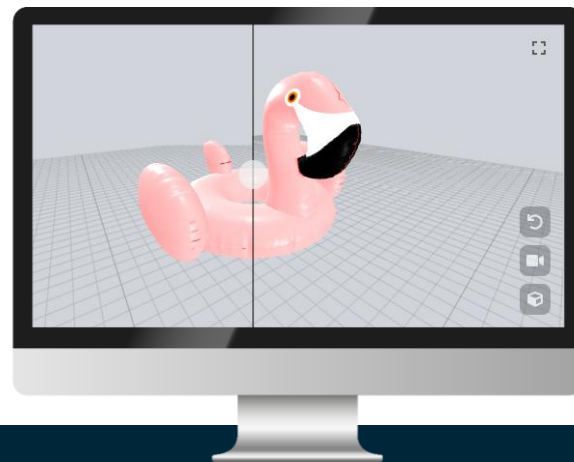
**Polygon
reduction**



**3D CMS
multi-connected**



**Texture & animation
preservation**



**The solution to all your
problems is AMAZ3D**



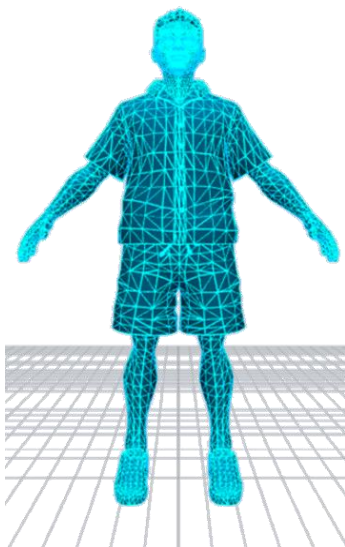
AMAZ3D **The all-in-one
optimization platform**

We reduce polygons / preserve accuracy

ORIGINAL OBJECT

9 MB

13,232 polygons



Original object LOD0

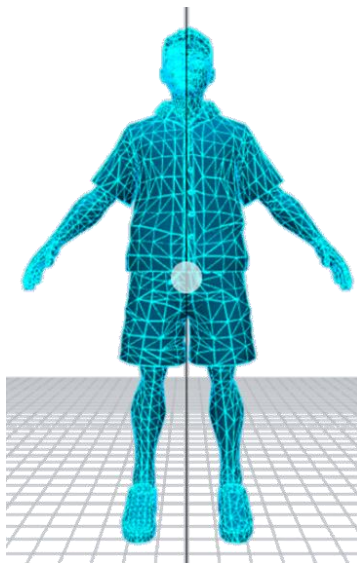
LOD: Level of Detail

AVERAGE OPTIMIZATION

AMAZ3D

5 MB

6,615 polygons



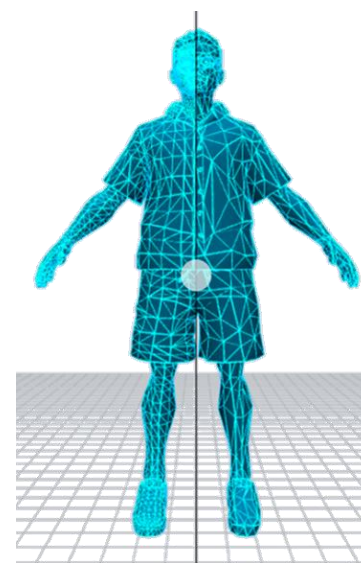
AMAZ3D object LOD1

ADVANCED OPTIMIZATION

AMAZ3D

3 MB

3,307 polygons



AMAZ3D object LOD2

Optimization Strategies

Rendering

- Frustum culling — don't render off-screen objects
- Occlusion culling — skip hidden geometry
- Instanced rendering for repeated objects
- Texture atlasing to reduce draw calls

Loading

- Lazy loading — load assets on demand
- Progressive asset streaming
- Draco + KTX2 compression
- Service workers for caching

Questions?

Let's discuss

Thank you for your attention

